

Cost Model: Codebase Audit

netsquid-qkd — 2026-08-03

Contents

1	Overview	2
2	Logic Flow	2
2.1	File dependency graph	2
2.2	Cost computation pattern in <code>cost_sweep.py</code>	2
2.3	Fibre distance calculation per protocol	3
3	SPD Cost Model	3
3.1	Linear model	3
4	Component Model	4
4.1	Hardware quantities per protocol	4
4.2	Physical rationale per protocol	4
4.3	Trusted BB84 relay-relay key security note	4
5	Total Cost Formula	5
6	Default Costs and Technology Presets	5
6.1	DEFAULT_COSTS	5
7	CLI Flags in <code>cost_sweep.py</code>	5
8	Database Schema and Cost Persistence	5
8.1	What is stored	5
8.2	What is not stored	6
9	Assumptions and Limitations	6
9.1	Classical communication is free	6
9.2	Fibre is Euclidean, not routed	7
9.3	BB84 mesh fibre	7
9.4	Cost and simulation models are not fully coupled	7
9.5	No operational expenditure	7
9.6	Source technology is fixed to QD	7
9.7	Source count is approximate for trusted BB84 backbone	8
10	Cross-Analysis: Simulation vs. Cost Model	8

10.1	Simulation parameters and their cost model counterparts	8
10.2	Inconsistency I: <code>node_loss_db</code> aggregates individually-costed components .	8
10.3	Inconsistency II: SPAD dark count rate not updated with detector class . .	9
10.4	Inconsistency III: source error rate not linked to QD physics	9
10.5	Inconsistency IV: stale <code>n_spd</code> in MDI network return dict	10
10.6	Inconsistency V: optical switch loss is pair-conditional; cost is per-relay . .	10
10.7	Inconsistency VI: TX-side coupling loss has no cost entry	10
10.8	Inconsistency VII: trusted BB84 backbone is simulated as unidirectional . .	10
10.9	Inconsistency VIII: dark count rate absent from SPD cost model	11
11	Quick Reference: Effective Costs at Key Configurations	11
11.1	SPAD detectors, QD source	11
11.2	SNSPD detectors, QD source	11

1 Overview

All monetary values are in GBP. The cost model has two components:

- `network/cost.py` — core module defining unit costs, technology presets, component count formulae, the SPD linear cost model, and the `total_cost` function.
- `scripts/network/cost_sweep.py` — analysis script that sweeps user count N at fixed relay count K , runs simulations for all three protocols, and plots deployment cost and cost-efficiency.

The SQLite database (`data/results.db`) stores fibre length from each simulation run but no cost figures. Cost is recomputed on demand by `cost_sweep.py` using the parameters passed at the time of analysis.

2 Logic Flow

2.1 File dependency graph

```
network/cost.py
    +-- defines: DEFAULT_COSTS, DETECTOR_TECH, SOURCE_TECH
               spd_cost_from_efficiency(eta)
               component_counts(N, K, protocol)
               total_cost(counts, total_fibre_km, detector_efficiency, ...)

network/bb84_network.py
    +-- run_bb84_network() -> returns dict including total_fibre_km
network/mdi_network.py
    +-- run_mdi_network() -> returns dict including total_fibre_km
network/trusted_bb84_network.py
    +-- run_trusted_bb84_network() -> returns dict including total_fibre_km

lib/db.py
    +-- network_results table: stores total_fibre_km (no cost columns)
    +-- p2p_results table:    stores per-pair physics results (no cost)

scripts/network/cost_sweep.py
    +-- imports cost.py; calls run_*_network(); calls component_counts()
       + total_cost() to compute cost; plots cost vs N, efficiency vs N,
       and marginal cost vs N
```

2.2 Cost computation pattern in `cost_sweep.py`

1. Run QKD simulation to obtain `res["total_fibre_km"]`.
2. Read `cfg["detector_efficiency"]` (set by `--detector-tech` preset or simulation config).
3. Call `component_counts(N, K, protocol)` for hardware quantities.
4. Call `total_cost(counts, fibre_km, det_eff, **cost_kw)`.

5. Read ["total_gbp"] from the result.

In `--cost-only` mode the simulation is skipped; fibre length is computed directly from Euclidean topology geometry via `_fibre_km.bb84(topo)` and `_fibre_km.mdi(topo)`.

2.3 Fibre distance calculation per protocol

BB84 direct mesh Sum of all $N(N-1)/2$ pairwise Euclidean distances. Models a fully meshed dark fibre deployment with one fibre per user pair.

MDI-QKD N user-relay links (Euclidean) plus $K(K-1)/2$ relay-relay backbone links (Euclidean). Computed by `_mdi_fibre_cost()` in `mdi_network.py`.

Trusted BB84 Identical topology to MDI: N user-relay links plus $K(K-1)/2$ relay-relay backbone links. The backbone carries quantum signals (relay-to-relay BB84) as well as classical key forwarding traffic.

All distances are straight-line Euclidean on a flat $A \times A$ km grid. No tortuosity or routing factor is applied.

3 SPD Cost Model

3.1 Linear model

Rather than assigning a fixed price per detector, SPD cost is modelled as a linear function of detector efficiency η , anchored at two real technology classes:

Key	Technology	Efficiency η	Cost per SPD (GBP)
SPAD	InGaAs SPAD, telecom-band (1550 nm)	0.20	£15,000
SNSPD	Superconducting nanowire, cryogenic (~ 2 K)	0.85	£100,000

Table 1: `DETECTOR_TECH` anchor points in `network/cost.py`.

The linear fit through these two points gives:

$$C_{\text{SPD}}(\eta) = m\eta + c$$

where

$$m = \frac{100,000 - 15,000}{0.85 - 0.20} = \frac{85,000}{0.65} \approx \text{£}130,769 \text{ per unit } \eta$$

$$c = 15,000 - m \times 0.20 \approx -\text{£}11,154$$

This is implemented in `spd_cost_from_efficiency(eta)`, which raises a `ValueError` for $\eta < 0.20$ (below the InGaAs SPAD baseline). The function is called internally by `total_cost`; there is no explicit per-SPD cost override available at the CLI level.

Spot checks: $\eta = 0.20 \rightarrow \text{£}15,000$; $\eta = 0.85 \rightarrow \text{£}100,000$; $\eta = 0.50 \rightarrow \text{£}54,231$.

4 Component Model

4.1 Hardware quantities per protocol

The function `component_counts(N, K, protocol)` returns the following:

Protocol	Sources	SPDs	BS	PBS	EOM	Switch	Links
BB84 (direct mesh)	N	$2N$	0	N	N	0	$N(N-1)/2$
MDI-QKD	N	$4K$	K	$2K$	0	K	$N + K(K-1)/2$
Trusted BB84	$N + K$	$2K$	0	K	K	K	$N + K(K-1)/2$

Table 2: Hardware component counts. N = users, K = relays. BS = 50:50 fibre coupler; PBS = polarising beam splitter; EOM = electro-optic modulator.

4.2 Physical rationale per protocol

BB84 direct mesh. Each user has one photon source (Alice) and a receiving module (Bob). Active basis choice is assumed: an EOM rotates the incoming polarisation to align with a fixed PBS, effectively switching which basis is measured. Bob components per user: 1 EOM + 1 PBS + 2 SPDs. Full mesh: one dark fibre per pair, hence $N(N-1)/2$ links.

MDI-QKD. Users supply sources; MDI relay nodes are passive Bell state measurement (BSM) stations — no source at a relay, no basis choice required. Each relay performs a full polarisation-resolving BSM:

- $1 \times$ 50:50 fibre coupler — Hong–Ou–Mandel interference.
- $2 \times$ PBS — one per output arm, for polarisation analysis.
- $4 \times$ SPDs — two per PBS (H and V outputs), giving complete Bell state discrimination between $|\Psi^+\rangle$ and $|\Psi^-\rangle$.
- $1 \times$ optical switch — routes photons from cross-cluster users to the correct relay for BSM.

Fibre links: N user-to-relay links plus $K(K-1)/2$ relay-to-relay backbone links.

Trusted BB84. Users connect to their assigned relay via direct BB84 links. The relay acts as Bob on all user-relay links, sharing a single detector array (1 EOM + 1 PBS + 2 SPDs) across all connected users. An optical switch routes each incoming user signal to the detector array in turn. Relays also run BB84 with one another over the backbone to establish inter-relay keys, requiring one photon source per relay (simplified from the strict $K-1$ sources per relay that a fully connected backbone would require). Total sources: $N + K$. No BSM hardware. Classical key forwarding (XOR masking) is authenticated but not costed (see Section 9).

4.3 Trusted BB84 relay-relay key security note

The classical message transmitted between relay nodes during key forwarding takes the form $K_{AR_1} \oplus K_{R_1R_2}$, where $K_{R_1R_2}$ was established by relay-to-relay QKD and is secret.

This is equivalent to a one-time pad: an in-flight attacker observing the classical channel cannot recover K_{AR_1} . The true trust assumption is not channel security but node security: a physically compromised relay node has access to all key material it has processed. MDI-QKD eliminates this vulnerability because relay nodes (Charlie) never hold any key material.

5 Total Cost Formula

$$C_{\text{total}} = N_s C_s + N_d C_{\text{SPD}}(\eta) + N_b C_b + N_{\text{pbs}} C_{\text{pbs}} + N_{\text{eom}} C_{\text{eom}} + N_{\text{sw}} C_{\text{sw}} + L_{\text{fibre}} C_f$$

where $N_s, N_d, N_b, N_{\text{pbs}}, N_{\text{eom}}, N_{\text{sw}}$ are the source, SPD, 50:50 BS, PBS, EOM, and optical switch counts from `component_counts`; $C_{\text{SPD}}(\eta)$ is the linear SPD cost model; L_{fibre} is total fibre in km; and remaining C_x are unit costs.

`total_cost()` returns a dict with `total_gbp`, `hardware_gbp`, `fibre_gbp`, and a `breakdown` sub-dict with per-category figures.

6 Default Costs and Technology Presets

6.1 DEFAULT_COSTS

Key	Component	Default (GBP)	Notes
<code>source_gbp</code>	Photon source (QD)	£150,000	QD SPS + cryostat, amortised
<code>bs_gbp</code>	50:50 fibre coupler	£1,000	HOM BS at MDI relay
<code>pbs_gbp</code>	Polarising BS	£500	BB84/TBB84 receivers; MDI arms
<code>eom_gbp</code>	EOM	£2,000	Active basis choice; BB84/TBB84
<code>switch_gbp</code>	Optical switch	£5,000	Cross-relay routing; MDI only
<code>fibre_per_km_gbp</code>	Installed fibre	£10,000/km	Dark fibre, installed

Table 3: `DEFAULT_COSTS` in `network/cost.py`. SPD cost is not listed here — it is always derived from detector efficiency via the linear model. All non-SPD values are indicative placeholders pending cost verification.

7 CLI Flags in `cost_sweep.py`

Priority order: `DEFAULT_COSTS` $\rightarrow$ `--detector-tech` preset $\rightarrow$ explicit CLI flag. Detector efficiency is taken from `cfg["detector_efficiency"]`, set by `--detector-tech` or the JSON config file.

8 Database Schema and Cost Persistence

8.1 What is stored

`network_results` — one row per network simulation run:

- Protocol, N , K , area, seed, runtimes, experiment label.

Flag	Type	Effect
<code>--detector-tech SPAD SNSPD</code>	preset	Sets η in sim; SPD cost derived from η
<code>--source-cost FLOAT</code>	GBP override	Replaces QD default source cost
<code>--bs-cost FLOAT</code>	GBP override	Replaces default 50:50 BS cost
<code>--pbs-cost FLOAT</code>	GBP override	Replaces default PBS cost
<code>--eom-cost FLOAT</code>	GBP override	Replaces default EOM cost
<code>--switch-cost FLOAT</code>	GBP override	Replaces default optical switch cost
<code>--fibre-cost FLOAT</code>	GBP/km override	Replaces default fibre cost

Table 4: Cost-relevant CLI flags. There is no `--spd-cost` or `--source-tech` flag; SPD cost is always derived from detector efficiency, and source technology is fixed to QD.

- `total_fibre_km`, `avg_pair_distance_km`, `cross_relay_ratio`.
- Key rate statistics: `avg_key_rate`, `std_key_rate`, `success_rate`, `min_key_rate`, `max_key_rate`.

`p2p_results` — one row per Monte Carlo runtime per pair: physical parameters, key rate, QBER, key length, status.

8.2 What is not stored

No cost column exists in either table. The unit costs used at analysis time (source, SPD efficiency, BS, PBS, switch, fibre) are not persisted alongside simulation results. A stored network row cannot be repriced from the database alone without knowing the cost parameters supplied at the time.

Proposed addition — add the following columns to `network_results` at insert time:

```
ALTER TABLE network_results ADD COLUMN source_tech TEXT;
ALTER TABLE network_results ADD COLUMN detector_tech TEXT;
ALTER TABLE network_results ADD COLUMN detector_efficiency REAL;
ALTER TABLE network_results ADD COLUMN source_cost_gbp REAL;
ALTER TABLE network_results ADD COLUMN spd_cost_gbp REAL;
ALTER TABLE network_results ADD COLUMN bs_cost_gbp REAL;
ALTER TABLE network_results ADD COLUMN pbs_cost_gbp REAL;
ALTER TABLE network_results ADD COLUMN switch_cost_gbp REAL;
ALTER TABLE network_results ADD COLUMN fibre_cost_per_km_gbp REAL;
;
ALTER TABLE network_results ADD COLUMN hardware_cost_gbp REAL;
ALTER TABLE network_results ADD COLUMN fibre_cost_gbp REAL;
ALTER TABLE network_results ADD COLUMN total_cost_gbp REAL;
```

9 Assumptions and Limitations

9.1 Classical communication is free

All classical communication costs — synchronisation channels, basis reconciliation, error correction, privacy amplification, authenticated key forwarding at trusted relays — are

assumed to be zero. Only quantum-layer hardware and quantum fibre links are costed. This is a standard simplification for quantum network techno-economic analysis at the early-stage modelling level.

9.2 Fibre is Euclidean, not routed

All link distances are straight-line Euclidean. Real fibre follows duct paths; a tortuosity factor of $1.2\text{--}1.5\times$ is typical in urban deployments. A `--tortuosity` flag (default 1.0) could scale all fibre distances before cost computation without affecting the physics simulation.

9.3 BB84 mesh fibre

The BB84 cost model counts $N(N-1)/2$ individual dark fibre runs, one per user pair. This is correct for a fully meshed passive dark-fibre network but unrealistic at scale; a WDM or switched architecture would replace individual fibres. This makes BB84 costs appear very large at high N , which is physically meaningful (mesh BB84 does not scale well) but should be interpreted with caution as a direct deployment cost estimate.

9.4 Cost and simulation models are not fully coupled

The cost model counts individual optical components (EOM, PBS, 50:50 BS, optical switch) at each node. The simulation, however, abstracts all receiver-side optics into a single `node_loss_db` parameter and a single `detector_efficiency` value. Loss contributions from the EOM, PBS, and switching fabric are not modelled separately in the physics layer — they are subsumed into the aggregate node loss figure. Similarly, the optical switch at trusted BB84 and MDI relays introduces insertion loss in practice, but this is not broken out as a distinct simulation parameter.

This means the cost model and the rate model are not tightly coupled: adding more components raises the modelled cost but does not automatically degrade the simulated key rate. The correct approach when refining component budgets is to update both the cost model here *and* the node loss parameter in the simulation config to reflect realistic insertion losses for the chosen hardware.

9.5 No operational expenditure

The model is purely capital expenditure (CapEx). For cryogenic technologies (SNSPD at $\sim 2\text{ K}$, QD at $\sim 4\text{ K}$) the operational cost of cryogenic cooling can rival the hardware CapEx over a 10-year lifetime at commercial electricity prices, and is entirely unmodelled here.

9.6 Source technology is fixed to QD

The simulation uses ideal single-photon sources throughout. For cost purposes, the source is assumed to be a semiconductor quantum dot (QD) system at £150,000 per node, reflecting cryogenic hardware and amortised cryostat cost. This value is a literature-derived estimate and should be verified. Use `--source-cost` to override when a revised figure is available.

9.7 Source count is approximate for trusted BB84 backbone

The model assigns K backbone sources (one per relay) for relay-to-relay QKD links. Strictly, a fully connected backbone with K nodes and bidirectional BB84 requires each relay to have $K-1$ sources (one per neighbour), giving $K(K-1)$ total backbone sources. The current approximation of 1 source per relay underestimates backbone hardware cost for $K > 2$. For the relay counts typical in this project ($K \leq 8$) the error is bounded but non-negligible at high K .

10 Cross-Analysis: Simulation vs. Cost Model

This section maps every simulation parameter in `lib/functions.py` and the network runner files against its counterpart (or absence) in the cost model, and documents all identified inconsistencies.

10.1 Simulation parameters and their cost model counterparts

Simulation parameter	Default value	Cost model entry	Notes
<code>fibre_loss_db_per_km</code>	0.20 dB/km	<code>fibre_per_km_gbp</code>	Physics and cost are independent; coupled only by total km
<code>init_loss</code>	0.15 (linear)	<i>none</i>	TX-side coupling loss; no cost component
<code>detector_efficiency</code>	0.85	<code>spd_cost_from_efficiency</code>	Fully coupled: η drives both rate and SPD cost
<code>dark_count_rate</code>	100 cps	<i>none</i>	Only η drives cost; dark count not linked to detector class
<code>node_loss_db</code>	2.0 dB	EOM + PBS + switch + BS	All RX-side components lumped into one scalar
<code>source_error_rate</code>	0.005	<i>none</i>	Not linked to QD assumption
<code>dephasing_rate</code>	0.0001	<i>none</i>	Physical noise; no hardware cost correlate
<code>bs_eff</code>	0.97 (MDI only)	<code>bs_gbp</code>	Physics and cost independent; both MDI-only
<code>SWITCH_LOSS_DB</code>	1.0 dB (MDI)	<code>switch_gbp</code>	Loss conditional; cost unconditional (see below)

Table 5: All simulation parameters against cost model entries. *None* indicates no cost model counterpart exists.

10.2 Inconsistency I: `node_loss_db` aggregates individually-costed components

Severity: high.

The simulation aggregates all receiver-side optical losses into a single `node_loss_db` parameter (default 2.0 dB, described in `configs/layer5_node_loss.json` as “FC/PC connectors + optical components at receiver”). The cost model, by contrast, prices each component individually: EOM, PBS, 50:50 BS (MDI), and optical switch.

Typical per-component insertion losses are:

- EOM (lithium niobate): 0.5–3.0 dB
- PBS (fibre-coupled): 0.1–0.5 dB
- Optical switch (MEMS or thermo-optic): 0.5–2.0 dB
- 50:50 fibre coupler (HOM BS): 0.1–0.3 dB (excess loss only; 3 dB splitting is accounted for by `bs_eff`)

For a BB84 or trusted BB84 receiver with EOM + PBS, the aggregate RX loss could plausibly reach 3.5–3.5 dB. For an MDI relay with 50:50 BS + PBS + switch, aggregate loss could reach 2.6–4.3 dB. The 2.0 dB default is therefore potentially too low, and adding components to the cost model without updating `node_loss_db` in the simulation configs will silently overstate performance while raising cost.

Recommended action: define per-component loss budgets and update `node_loss_db` in the relevant JSON configs to reflect the actual hardware stack being costed.

10.3 Inconsistency II: SPAD dark count rate not updated with detector class

Severity: high.

When `--detector-tech SPAD` is passed, the simulation correctly sets $\eta = 0.20$, but `dark_count_rate` remains at its default of 100 cps. This is unrealistic: InGaAs SPADs have dark count rates of approximately 1,000–50,000 cps, compared to 1–100 cps for SNSPDs. Simulating SPAD cost with SNSPD-tier dark counts significantly overstates SPAD key rate performance.

Recommended action: couple `dark_count_rate` to `--detector-tech` in `cost_sweep.py`, e.g. SPAD $\rightarrow$ 10,000 cps, SNSPD $\rightarrow$ 100 cps. These values should themselves be verified against the literature.

10.4 Inconsistency III: source error rate not linked to QD physics

Severity: low–medium.

The simulation default `source_error_rate = 0.005` (0.5%) is a fixed value with no explicit connection to the QD source assumption used in the cost model. QD single-photon sources have finite multi-photon emission probability ($g^{(2)}(0) > 0$) that contributes to QBER. A value of 0.5% is plausible for a high-quality QD device but is not derived from a specific QD literature value. There is currently no mechanism to propagate source technology choice into the simulation physics.

Recommended action: note the assumed value in the simulation config or docstring, and cite the QD literature source when verifying the cost figure.

10.5 Inconsistency IV: stale `n_spd` in MDI network return dict

Severity: low (documentation only).

`mdi_network.run_mdi_network()` returns a dict that includes `"n_spd": 2 * topo.K`. The cost model now uses $4K$ SPDs per MDI deployment (full polarisation-resolving BSM). The return dict value is not used in cost computation — cost is computed via `component_counts` independently — but the stale field is misleading if the return dict is used for diagnostics or future analysis.

Recommended action: update the return dict to `"n_spd": 4 * topo.K`.

10.6 Inconsistency V: optical switch loss is pair-conditional; cost is per-relay

Severity: low (by design, but worth noting).

In `mdi_network.py`, the optical switch insertion loss (`SWITCH_LOSS_DB = 1.0 dB`) is applied only to cross-cluster pairs, i.e. pairs whose two users are assigned to different relays. In-cluster pairs (both users on the same relay) incur no switch loss. The cost model, however, charges for one switch per MDI relay unconditionally.

This is physically correct — the switch must be installed at every relay regardless of which pairs use it — but it creates an asymmetry: the cost is borne by the network as a whole while the performance penalty is borne only by cross-cluster pairs. Networks with high K (many relays, low cross-cluster ratio) will show a larger switch cost fraction relative to the switch-induced rate penalty.

10.7 Inconsistency VI: TX-side coupling loss has no cost entry

Severity: negligible.

`init_loss = 0.15` models source-to-fibre coupling efficiency (15% loss). The coupling hardware — fibre collimator, circulator, or pigtailed connector at the transmitter — has no corresponding entry in the cost model. Unit costs for such components are typically £100–£500 each, small relative to the QD source at £150k. Omitting them is acceptable but should be acknowledged.

10.8 Inconsistency VII: trusted BB84 backbone is simulated as unidirectional

Severity: medium.

`trusted_bb84_network.py` simulates $K(K-1)/2$ backbone links, one per relay pair, each running a single unidirectional BB84 instance. The key generated on link (j, k) is shared between relay j and relay k , and is divided among all cross-relay end-to-end pairs that traverse that backbone segment (key dilution). For full bidirectional key generation — where j can independently forward a key to k and k can forward a separate key to j — each relay would need two BB84 sessions per neighbour (doubling backbone source count and rate). The simulation implicitly treats the single key as shared, which may overstate backbone throughput for dense cross-relay traffic.

10.9 Inconsistency VIII: dark count rate absent from SPD cost model

Severity: low.

The linear SPD cost model $C_{\text{SPD}}(\eta) = m\eta + c$ depends only on detector efficiency. Dark count rate is a second major performance parameter that also varies strongly with technology class: SNSPDs achieve $\sim 1\text{--}100$ cps while InGaAs SPADs are typically $\sim 1,000\text{--}50,000$ cps. Both efficiency and dark count rate are correlated with cost in practice, but the model captures only the efficiency dimension. This means two detector technologies with the same η but different dark count rates would be assigned identical costs, which may not reflect reality for technologies at intermediate efficiencies on the linear model.

11 Quick Reference: Effective Costs at Key Configurations

11.1 SPAD detectors, QD source

Component	Cost (GBP)	Origin
Source (QD, per node)	£150,000	DEFAULT_COSTS
SPD (SPAD, per detector)	£15,000	spd_cost_from_efficiency(0.20)
EOM (BB84/TBB84 receivers)	£2,000	DEFAULT_COSTS
PBS	£500	DEFAULT_COSTS
50:50 BS (MDI relay)	£1,000	DEFAULT_COSTS
Optical switch (MDI relay)	£5,000	DEFAULT_COSTS
Fibre (per km, installed)	£10,000	DEFAULT_COSTS
Detector efficiency η	0.20	DETECTOR_Tech["SPAD"]

Table 6: Run with `--detector-tech SPAD`. Optical switch applies to both MDI and trusted BB84 relays.

11.2 SNSPD detectors, QD source

Component	Cost (GBP)	Origin
Source (QD, per node)	£150,000	DEFAULT_COSTS
SPD (SNSPD, per detector)	£100,000	spd_cost_from_efficiency(0.85)
EOM (BB84/TBB84 receivers)	£2,000	DEFAULT_COSTS
PBS	£500	DEFAULT_COSTS
50:50 BS (MDI relay)	£1,000	DEFAULT_COSTS
Optical switch (MDI relay)	£5,000	DEFAULT_COSTS
Fibre (per km, installed)	£10,000	DEFAULT_COSTS
Detector efficiency η	0.85	DETECTOR_Tech["SNSPD"]

Table 7: Run with `--detector-tech SNSPD`.